

PROJECT TECHNICAL DOCUMENTATION

ContractSentinel

AI Contract-Risk Analyzer

An autonomous, local-first LangGraph agent that reads a legal contract, identifies and scores risky clauses on real legal evidence, drafts safer rewrites, and compiles a professional risk report.

Yash Joshi

Backend Implementation & Agent Pipeline

Team: Yash Joshi (Backend) · Bansil (Frontend)

Document version 1.0 · Generated 7 August 2026

Table of Contents

1. Executive Summary
2. Problem Statement & Motivation
3. Project Objectives
4. System Overview — How It Works
5. System Architecture: The 7-Node Pipeline
6. Node-by-Node Deep Dive
7. Conditional Routing & Evidence Handling
8. Internal State & Data Model
9. Technology Stack
10. Backend Platform — API, Persistence, Delivery
11. Security Posture
12. Frontend & User Interface
13. Development Methodology
14. Feature History
15. Challenges & Solutions
16. Current Status — Done vs. In Progress
17. Future Work & Roadmap
18. Running the Project Locally
19. Conclusion

1. Executive Summary

ContractSentinel is an AI-powered system that automatically analyzes legal contracts to find potential risks and suggest improvements. A user uploads a PDF or DOCX contract; the system splits it into individual clauses, retrieves supporting legal evidence for each clause, validates which clauses are genuinely worth flagging, assigns a Low / Medium / High risk level with a written rationale, drafts a safer rewrite for the risky clauses, and compiles a branded PDF report that is delivered to the user's own Google Drive and emailed to them.

The distinguishing design choice is that the entire analysis runs **locally**. The generative model (Qwen3:8b) and the embedding model (BGE-M3) are both served through a local Ollama runtime, so confidential contract text never leaves the machine — a direct answer to the privacy problem that prevents many organisations from using cloud AI tools on their contracts.

Architecturally, the analysis is a **fixed seven-node LangGraph state machine** with exactly two conditional edges, defined up front in a project “constitution” and held immutable throughout development. Around that pipeline sits a real software product: a FastAPI backend with live progress streaming, durable SQLite persistence that survives crashes, per-user data isolation, a full authentication and security layer, and delivery integrations to Google Drive and Gmail.

Status at this review. The backend — the complete pipeline, the API, persistence, security and delivery — is implemented and covered by an extensive automated test suite (930+ backend tests). A set of user-interface pages has been designed in Next.js. Wiring those pages into a single, polished end-to-end product is the work that remains.

2. Problem Statement & Motivation

Contracts govern almost every business relationship, yet reviewing them well is slow, expensive, and error-prone:

- **Manual review does not scale.** A careful clause-by-clause review takes a trained reviewer hours per contract, and every new contract starts from scratch.
- **Risky clauses are easy to miss.** Indemnification, limitation of liability, termination, auto-renewal and governing-law clauses carry real exposure, and a single missed clause can be costly.
- **Small teams often have no reviewer at all.** Startups and individuals frequently sign contracts without any legal check.
- **Cloud AI tools leak confidential data.** Sending a contract to a hosted LLM means the document — often highly sensitive — leaves the organisation's control.

ContractSentinel targets exactly this gap: an automated reviewer that is fast, produces explainable and evidence-grounded findings, and runs entirely on local infrastructure so that privacy is preserved by construction.

3. Project Objectives

#	Objective	Description
1	Automate clause-level risk analysis	Take a raw PDF/DOCX contract and automatically detect, classify and score every risky clause.
2	Ground findings in evidence	Retrieve supporting legal knowledge for each clause so that verdicts are explainable rather than opaque guesses.
3	Suggest safer rewrites	Draft an improved redline for high-risk clauses and compile a professional, shareable report.
4	Stay private and local-first	Run all models locally through Ollama so that confidential contracts never leave the machine.
5	Behave like a real SaaS	Provide authentication, per-user data isolation, a dashboard, report history, and delivery to the user's own Drive and email.
6	Be rigorously engineered	Follow a strict spec-driven workflow with a fixed architecture and a large automated test suite.

4. System Overview — How It Works

From the user's perspective the product is a simple four-step flow:

Step	What happens
Upload	The user signs in and uploads a PDF or DOCX contract through the web app.
Analyze	The seven-node AI pipeline splits the document into clauses, retrieves evidence, validates the genuinely risky clauses, and scores each one.
Redline	For risky clauses, the system drafts a suggested safer rewrite with a rationale; clean clauses are marked and skipped.
Deliver	A branded PDF report with the full evidence trail is saved to the user's Google Drive and emailed to them.

Every clause that survives validation ends up with one of three risk levels:

Level	Meaning
HIGH	Serious exposure — needs a rewrite before signing.
MEDIUM	Worth reviewing or negotiating.
LOW	Acceptable, standard wording.

5. System Architecture: The 7-Node Pipeline

The core of ContractSentinel is a **LangGraph** StateGraph with exactly **seven sequential nodes** and exactly **two conditional edges**. This shape is fixed by the project constitution (§2, “Fixed Architecture Rule”) and is treated as immutable: every feature is built around this graph rather than by adding new nodes. Each node returns only the state keys it updates (a partial update), and large content such as raw document text is stored by reference (a file path) because LangGraph checkpoints the state after every step.

#	Node	Responsibility
1	IngestAgent	Parse the PDF/DOCX into text (OCR fallback for scans).
2	ClauseSplitterAgent	Segment the document into discrete, typed clauses.
3	CRAG Retrieval	Retrieve supporting legal evidence per clause (conditional).
4	Self-RAG Validation	Decide which clauses are genuinely worth flagging.
5	RiskScoreAgent	Assign Low / Medium / High risk with a rationale.
6	route_on_risk → Redline / Skip	Draft a safer rewrite, or skip (conditional).
7	ReportAgent	Compile the final report plus the evidence trail.

Post-terminal delivery. After the graph reaches its terminal END, a separate delivery step (not a graph node) pushes the report to Google Drive and sends the notification email. Keeping delivery outside the graph preserves the fixed seven-node shape.

Two models, kept strictly separate. The generative model **Qwen3:8b** and the embedding model **BGE-M3** are always distinct model objects served through Ollama; the embedding model is never substituted for the generative one (constitution §8).

6. Node-by-Node Deep Dive

Node 1 — IngestAgent

Parses the uploaded contract into raw text. PDFs are read with PyMuPDF and DOCX files with python-docx; if a page yields no extractable text (a scanned document), the agent falls back to Tesseract OCR. Because uploaded files are encrypted at rest, IngestAgent first decrypts the file to a short-lived temporary file, parses it, and then removes the temp file — legacy plaintext uploads are still tolerated.

Node 2 — ClauseSplitterAgent

Segments the parsed document into discrete clauses. A fast regex pre-pass proposes candidate boundaries, then the LLM refines the boundaries and assigns each clause a type (e.g. indemnification, liability, termination). For very large documents the expensive LLM refinement is size-gated so performance stays predictable.

Node 3 — CRAG Retrieval (Corrective RAG)

For each clause the agent computes a BGE-M3 embedding and searches a local FAISS vector knowledge base of legal reference material, producing a retrieval-confidence score. If confidence is at or above the configured threshold (0.73) it uses the local knowledge base; if confidence is below the threshold it falls back to a live web search. Evidence is merged per clause regardless of which path produced it. This confidence-based branch is the first of the two conditional edges.

Node 4 — Self-RAG Validation

Guards against false alarms. Against the merged evidence the agent runs a relevance check, an ISREL (is-relevant) check and an ISSUP (is-supported / worth-flagging) check, retrying on failure up to a configured maximum. A clause is either discarded (never shown to the user) or validated. A **recall floor** protects genuinely high-risk clause types from being dropped even when the evidence signal is thin — this raised measured recall from about 64% to 100% on the evaluation set.

Node 5 — RiskScoreAgent

Assigns each validated clause a risk level of Low, Medium or High together with a written rationale explaining the judgement. LLM calls here are pinned to a deterministic temperature so results are stable across runs.

Node 6 — route_on_risk → RedlineAgent / SkipRedline

The second conditional edge. Risky clauses are routed to the RedlineAgent, which drafts safer replacement wording; clauses without risk are routed to SkipRedline and simply marked clean.

Node 7 — ReportAgent

Assembles the final structured report: the validated findings, their risk levels and rationales, the suggested rewrites, and the full evidence trail. The report is rendered to a branded PDF for delivery.

7. Conditional Routing & Evidence Handling

Only two transitions in the entire graph are conditional; every other transition is a plain linear edge. This deliberate constraint keeps the pipeline easy to reason about, test and checkpoint.

- **Edge 1 — CRAG confidence routing.** Per clause: retrieval confidence ≥ 0.73 uses the local FAISS knowledge base; confidence < 0.73 triggers a live web-search fallback. Evidence from either path is merged before validation.
- **Edge 2 — route_on_risk.** After scoring: a risky clause goes to the RedlineAgent for a rewrite; a clean clause goes to SkipRedline.

All tunable thresholds — the 0.73 CRAG cutoff, the Self-RAG retry limits and pass/fail criteria — live as named constants in a single shared configuration module rather than being hardcoded inside node logic, so they can be tuned against real sample contracts without touching the pipeline code (constitution §3).

8. Internal State & Data Model

The project follows a clear typing convention (constitution §4). The LangGraph internal state uses a lightweight TypedDict (ContractState) with no runtime-validation overhead, while everything that crosses a system boundary — HTTP requests and responses, MCP payloads, file and database I/O — uses **Pydantic** models for runtime validation. The two are never mixed inside the graph.

- **Partial updates** — each node returns only the keys it changed, avoiding race conditions and redundant writes.
- **State minimality** — large content (raw document text, the PDF itself) is held as a file-path reference, never embedded in state, because state is checkpointed after every step.
- **Spec-first change rule** — if implementation reveals that the state schema must change, the schema specification is updated first, with a written rationale, before any code change.

9. Technology Stack

Every dependency maps to a specific node or architectural layer; the stack favours self-hostable components and adds no paid API beyond the local Ollama runtime.

Layer	Technologies	Serves
Orchestration	LangGraph, langgraph-checkpoint-sqlite, langchain-core	All 7 nodes; graph wiring & checkpointing
LLM & Embeddings	Ollama, Qwen3:8b (generative), BGE-M3 (embeddings)	Generation & retrieval, kept separate
Retrieval	FAISS, NumPy, DuckDuckGo search	CRAG node (local KB + web fallback)
Document parsing	PyMuPDF, python-docx, Tesseract OCR	IngestAgent
API & data	FastAPI, Server-Sent Events, SQLite, Alembic, Pydantic	Runner, API surface, persistence
Security & delivery	cryptography (Fernet), bcrypt, JWT, MCP SDK, reportlab	Auth, encryption, PDF, Drive/Gmail delivery
Frontend	Next.js 14, TypeScript, Tailwind CSS, Recharts	Web application & dashboards

The target backend runtime is **Python 3.11** — chosen for LangGraph compatibility and stable support across FAISS and the OCR libraries.

10. Backend Platform — API, Persistence, Delivery

Beyond the AI pipeline, ContractSentinel is a complete backend application.

API layer (FastAPI)

- `POST /api/analyze` — accepts an uploaded contract and starts an analysis job (returns 202 Accepted).
- `GET /api/jobs`, `/api/jobs/{id}`, `/api/dashboard` — job listings and aggregate metrics.
- `GET /api/jobs/{id}/events` — live progress streamed over Server-Sent Events as the pipeline advances through each node.
- `GET /api/jobs/{id}/report` — the final structured report.

- Authentication endpoints for sign-in, profile, password change and password reset.

Persistence

- A **SQLite job store** plus the LangGraph **SQLite checkpointer** make analysis durable: if the process is killed mid-run, the job resumes from the last completed node rather than restarting.
- Schema changes are managed with **Alembic** migrations (current head 0008).
- **Per-user data isolation** — every data read is scoped to the owning account; there is no cross-account visibility.

Delivery

A dedicated delivery layer uses MCP (Model Context Protocol) servers to upload the branded PDF report to the user’s own Google Drive (into a ContractSentinel folder, with human-readable report names) and to email it via Gmail. If a user has not connected their Drive, analysis still runs and the report is still emailed — the Drive step is simply skipped, and a report is never written to another user’s Drive.

11. Security Posture

Security was developed as a dedicated hardening phase and is a substantial part of the backend work.

Area	What was implemented
Authentication	Email + password (bcrypt with a SHA-256 pre-hash); an HS256 JWT held in an httpOnly cookie; a sliding 30-minute idle timeout with an 8-hour absolute cap.
Session invalidation	A server-side session_epoch enables ‘log out everywhere’, ends other sessions on password change, and invalidates sessions on password reset.
Brute-force defense	Per-IP rate limiting plus a persisted per-account lockout.
Forgot password	An emailed, time-limited, single-use reset link; tokens are stored hashed (HMAC-SHA256); the flow never discloses whether an email exists.
Encryption at rest	Fernet (AES-128-CBC + HMAC-SHA256) encrypts stored Google OAuth tokens and uploaded contract files, through one key-management seam.
Prompt-injection defense	Untrusted contract and evidence text is wrapped in per-call-nonce delimiters inside a user message, with trusted instructions kept in a separate system message, across all generative LLM calls.
Transport & headers	Strict security headers (X-Content-Type-Options, X-Frame-Options: DENY, a Content-Security-Policy, Referrer-Policy, COOP, Permissions-Policy); HSTS is available and TLS-gated.
Upload hardening	Magic-byte validation rejects a file whose real content does not match its extension (PDF must start with %PDF, DOCX must be a ZIP).

Known follow-ups (deferred): TLS termination at the edge, publishing the Google OAuth app to production, encryption at rest for the generated reports and parsed text, and dependency scanning in CI.

12. Frontend & User Interface

The frontend is a Next.js 14 + TypeScript + Tailwind application. The following pages have been designed; connecting them into a single polished end-to-end flow against the live backend is the main remaining work.

Page	Purpose
Landing	Marketing home page with product hero and overview.
Login / Auth	Sign-in, forgot-password and reset screens.
Upload	Drag-and-drop contract upload.
Processing	Live view of the pipeline's progress through each node.
Report	Analysis workspace showing findings, risk levels and suggested rewrites.
Dashboard	KPIs, a risk gauge and recent activity.
Contracts	Searchable, filterable history of past reports.
Settings	Profile and security (password) tabs.
Integrations	Google Drive and Gmail connection status.

13. Development Methodology

ContractSentinel is built with a strict **spec-driven workflow** defined in a project constitution. For every feature the sequence is fixed:

- **spec.md** — requirements: what the feature must do.
- **plan.md** — technical design: how it will be implemented.
- **tasks.md** — numbered implementation steps.
- **Implementation** — coding against the tasks.

No source file may be written until that feature's spec and plan exist and are approved. Every artifact must additionally pass a dedicated **spec-reviewer** review gate before the next stage begins. Development follows **TDD where practical** — tests are written and confirmed failing first, and a failing test is fixed by fixing the code, never by weakening the test. Each feature is built on its own `feature/0XX` branch and fast-forward merged into `main` only once its tests pass. The result is a large, coherent system: over **930 backend tests** and roughly **240 frontend tests**, with the architecture governed by explicit written amendments rather than ad-hoc change.

14. Feature History

The project was delivered as a sequence of small, reviewed features, all merged to `main`:

Features	Area	Delivered
001–009	State schema + the seven pipeline nodes	Core engine

Features	Area	Delivered
010–012	MCP Drive/Gmail delivery, runner + FastAPI + SSE, SQLite persistence	Platform
013–024	Next.js frontend: design system, upload, processing, report, dashboard, contracts, settings, integrations	Frontend
014 / 019	Single-user auth → per-user data isolation (multi-tenant)	Access
020–022	User profile, report history, Analysis Workspace redesign	UX
025–029	Pipeline latency levers, determinism/variance harness	Performance
026–027	Offline evaluation harness + Self-RAG recall floor	Accuracy
030–031	Professional PDF + branded email; per-user Google Drive	Delivery
032–038	Security tiers: token & file encryption, session hardening, rate-limit, prompt-injection defense, forgot-password	Security

15. Challenges & Solutions

Running an LLM on limited local hardware

A 6 GB GPU cannot hold large models. The project settled on Qwen3:8b with a roughly 70/30 GPU/CPU split and model pre-warming; disabling the model's 'thinking' mode on the structured-output calls cut a full run from about seven minutes to around ninety seconds.

Controlling LLM accuracy and noise

Early on, the validator sometimes discarded genuinely risky clauses. An offline evaluation harness was built to measure precision and recall on a gold set, and a Self-RAG recall floor was added — lifting recall from about 64% to 100% while keeping precision high.

Keeping confidential data private

Because contracts are sensitive, all inference runs locally and uploaded files are encrypted at rest with Fernet; nothing is sent to a cloud LLM.

Keeping a large system coherent

A spec-driven workflow with a mandatory review gate on every artifact, a fixed architecture, and TDD kept a multi-feature system consistent and fully tested.

16. Current Status — Done vs. In Progress

Area	Status	Notes
Backend pipeline (7 nodes)	Complete	Implemented and tested end-to-end.
API & persistence	Complete	FastAPI + SSE; durable SQLite; per-user isolation.
Security & delivery	Complete	Auth, encryption, rate-limit; Drive + Gmail.
Frontend UI pages	In progress	Pages designed; being wired to the live backend.
Full end-to-end integration	Planned	Polished upload→report demo; TLS; deploy.

17. Future Work & Roadmap

- **Now** — the backend is complete: pipeline, API, persistence, security and delivery, all tested.
- **Next** — finish the UI: wire every designed page to the live backend for a complete upload-to-report experience.
- **Then** — integrate end-to-end: polish the demo, add TLS termination, and publish the Google OAuth app to production.
- **Later** — evaluate and scale: grow the legal knowledge base, measure accuracy on a larger corpus, and continue tuning latency.

18. Running the Project Locally

Prerequisites. Ollama running with the models pulled (`ollama pull qwen3:8b` and `ollama pull bge-m3`); a Python virtual environment for the backend; and Node dependencies installed for the frontend.

Backend (port 8000, from backend/):

```
# apply DB migrations (first run / after pulling)
.venv/Scripts/python.exe -m alembic upgrade head

# start the API (local HTTP dev)
AUTH_COOKIE_SECURE=False .venv/Scripts/python.exe -m uvicorn app.api.main:app --host 127.0.0.1 --port 8000
```

Frontend (port 3000, from frontend/):

```
npm run dev # then open http://localhost:3000
```

The frontend's `.env.local` must set `NEXT_PUBLIC_API_PROVIDER=real` so it proxies to the backend on port 8000. Tests run with `pytest -q` (backend) and `npm run test` (frontend).

19. Conclusion

ContractSentinel demonstrates a complete, working AI backend for automated contract-risk analysis: a fixed seven-node LangGraph pipeline that ingests a contract, retrieves evidence, validates and scores risky clauses, drafts safer rewrites, and produces a professional report — all running locally to protect confidential data, and all wrapped in a real software platform with authentication, durable persistence, per-user isolation, strong security and integrated delivery. The engine and platform are implemented and extensively tested; a set of user-interface pages has been designed. The remaining work is to connect those pages into a single, polished end-to-end product — a well-defined next step on a solid, thoroughly engineered foundation.